Lista zadań nr 3

Zadanie 1.

Złożenie funkcji f i g definiujemy (jak pamiętamy z przedmiotu „Logika dla informatyków”) jako funkcję $x \mapsto f(g(x))$. Zdefiniuj dwuargumentową funkcję `compose`, której wynikiem jest złożenie (jednoargumentowych) funkcji przekazanych jej jako argumenty. Używając modelu podstawieniowego, prześledź wykonanie wyrażeń:

```
compose square inc 5
compose inc square 5
```

Zakładamy, że funkcja `square` oblicza kwadrat swojego argumentu, natomiast `inc` – wartość argumentu powiększoną o 1.

Zadanie 2.

Odwracanie kolejności elementów listy można zaimplementować używając funkcji `fold_right` w następujący sposób:

```
let fold_right_rev xs =
  List.fold_right (fun y ys -> ys @ [y]) xs []
```

Taka implementacja posiada jednak poważną wadę. Zaobserwuj ją przy użyciu następującego wyrażenia:

```
# List.length (fold_right_rev (List.init 10000 Fun.id))
```

Jak wiele `List.cons`-ów tworzy ta funkcja dla listy wejściowej długości n? Ile z nich to nieużytki?

Zadanie 3. (2 pkt)

Wykorzystując funkcję `fold_tree` z wykładu, zdefiniuj następujące funkcje:

- `tree_product t` – iloczyn wszystkich wartości występujących w drzewie,

- `tree_flip t` – odwrócenie kolejności: zamiana lewego i prawego poddrzewa wszystkich węzłów w drzewie,
- `tree_height t` – wysokość drzewa (liczba węzłów na najdłuższej ścieżce od korzenia do liścia),
- `tree_span t` – para złożona z wartości skrajnie prawego i skrajnie lewego węzła w drzewie (czyli najmniejszej i największej wartości w drzewie BST),
- `flatten t` – lista wszystkich elementów występujących w drzewie, w kolejności infiksowej. Kolejność ta polega na tym, że elementy drzewa posiadającego w korzeniu węzeł listuje się zaczynając od elementów lewego poddrzewa (w kolejności infiksowej), po których następuje wartość węzła, a następnie elementy prawego poddrzewa (w kolejności infiksowej). Ze względu na własność BST, lista `flatten t` dla drzewa BST `t` jest posortowana.

Zadanie 4. (2 pkt)

Implementacja funkcji `flatten` z zadania 3 posiada poważną wadę – tworzy ona duże ilości nieużytków oraz wykonuje nadmiarowe obliczenia. Tę wadę można szczególnie dobrze zaobserwować na przykładzie drzew, które „rosną tylko w lewo”:

```
let left_tree_of_list xs =
  List.fold_left (fun t x -> Node (t, x, Leaf)) Leaf xs
let test_tree = left_tree_of_list (List.init 20000 Fun.id)
```

Napisz inną implementację `flatten`, która nie posiada tej wady. Nie używaj funkcji `append` (ani operatora `@`)!

Wskazówka: zaimplementuj najpierw dwuargumentową funkcję `flat_append t xs`, której wynikiem jest lista elementów `t` w kolejności infiksowej scalona z listą `xs`. Przykład:

```
# flat_append t [10; 11]
- : int list = [2; 5; 6; 8; 9; 10; 11]
```

Zadanie 5.

Narysuj, jak w pamięci reprezentowane jest drzewo `t` zdefiniowane poniżej:

```
let t =
  Node (Node (Leaf, 2, Leaf),
        5,
        (Node (Node (Leaf, 6, Leaf)),
               8,
               (Node (Leaf, 9, Leaf))))
```

Pokaż, jak będzie wyglądał stan pamięci po wykonaniu usunięcia z drzewa `t` wartości 8. Które fragmenty drzewa `t` są współdzielone między drzewem `t` i `delete_bst 8 t`?

Zadanie 6.

Słownik to struktura danych przyporządkowująca kluczom pewnego typu danych wartości innego typu. Rozważmy następującą sygnaturę słownika:

```
module type DICT = sig
  type ('a, 'b) dict
  val empty : ('a, 'b) dict
  val insert : 'a -> 'b -> ('a, 'b) dict -> ('a, 'b) dict
  val remove : 'a -> ('a, 'b) dict -> ('a, 'b) dict
  val find_opt : 'a -> ('a, 'b) dict -> 'b option
  val find : 'a -> ('a, 'b) dict -> 'b
  val to_list : ('a, 'b) dict -> ('a * 'b) list
end
```

Zaimplementuj moduł `ListDict` implementujący tę sygnaturę przy użyciu list par kluczy i wartości.

Zadanie 7.

Zaproponuj alternatywną implementację sygnatury `DICT`, wykorzystującą drzewa przeszukiwań binarnych:

```
module TreeDict : DICT = struct
  type ('a, 'b) dict = Leaf | Node of ('a, 'b) dict * 'a * 'b *
      ('a, 'b) dict
  ...
end
```

Wskazówka do implementacji `remove`: Aby stworzyć drzewo przeszukiwań binarnych z którego usunęliśmy korzeń, najlepiej znaleźć (jeśli istnieje) najmniejszy element większy od tego korzenia.

Zadanie 8.

Używając sygnatury słownika `DICT`, zaimplementuj funkcję `freq` w funktorze `Freq` uzupełniając kod poniżej:

```
module Freq (D : DICT) = struct
  let freq (xs : 'a list) : ('a * int) list = ...
end
```

Wartością zwracaną powinna być lista par, których pierwszym elementem są elementy listy wejściowej `xs`, a drugim – liczba wystąpień tego elementu w tej liście.

Wykorzystując funktor `Freq`, napisz funkcję `char_freq`, obliczającą liczbę wystąpień znaków w ciągu typu `string`. Aby zamienić wartość `string` na listę znaków, możesz wykorzystać funkcję:

```
let list_of_string s = String.to_seq s |> List.of_seq
```

Wykorzystaj dowolny z modułów `ListDict` lub `TreeDict`.

Zadanie 9. (2 pkt)

Rozważmy następującą sygnaturę zawierającą pojedynczą funkcję:

```
module type TESTS = sig
  val run_tests : unit -> unit
end
```

Zaimplementuj funktor sprawdzający, czy zadana implementacja słownika spełnia następujące własności:

- Pusty słownik nie zawiera żadnego elementu.
- Element wstawiony do dowolnego słownika znajduje się w wynikowym słowniku. Sprawdź w osobnych testach zarówno `find`, jak i `find_opt`.
- Po wstawieniu do dowolnego słownika elementu, a następnie jego usunięciu, wynikowy słownik nie zawiera tego elementu.

- Dla dowolnego słownika, wszystkie elementy zwrócone przez `to_list` zawierają się w tym słowniku.

Aby umożliwić generowanie losowych słowników, zaimplementuj wewnątrz funktora funkcję `of_list`, tworzącą słownik z zadanej listy par. Sprawdź swoje testy na dowolnym z modułów `ListDict` lub `TreeDict`.